Chapter 7: Generative AI & De Novo Drug Design

Every model in Chapters 5 and 6 solves the **forward problem**: given a molecule, predict a property — hERG liability (Chapter 5), aqueous solubility (Chapter 6). This chapter solves the **inverse problem**: given a desired property, produce a molecule. Sanchez-Lengeling and Aspuru-Guzik (2018), surveying the field's shift from screening to design, frame the distinction precisely: forward models are cheap to evaluate but only ever score candidates someone already thought to propose, while inverse (generative) models search the space of possible structures directly, using a forward model's predictions as the signal that tells the search where to go. This chapter builds exactly that pairing: Chapter 5′s reward-oracle methodology, reapplied to a new target, becomes the compass a generative model uses to explore chemical space it was never shown during pretraining.

7.1 Inverse Molecular Design

A generative model for molecules faces a constraint forward models never do: its output has to be a real, buildable chemical structure, not just a scored one. Sanchez-Lengeling and Aspuru-Guzik (2018) organize inverse design methods along the representation they generate in (strings, graphs, or 3D geometries — Sections 7.2-7.4 cover one generative *model class* for each) and note that every one of them inherits the same three-part success criterion, later formalized as a standard benchmark by Brown et al. (2019) in GuacaMol: a generated output is only useful if it is

- **valid** — parses to a real molecule under the representation's own rules (a SMILES string with balanced rings and legal valences; a SELFIES string, per Chapter 2, is valid *by construction*),
- **novel** — not simply a memorized copy of a training-set molecule, and
- **synthetically accessible** — buildable by a medicinal chemist, not merely valid on paper.

These three criteria are in tension with each other by design: a model that only ever reproduces training molecules scores perfectly on validity and synthesizability but contributes nothing novel; a model that ignores the training distribution can generate valid nonsense — structures RDKit parses without complaint but no chemist would attempt to synthesize. Section 7.6′s hands-on project reports all three axes — validity, novelty (the fraction of valid generations whose canonical SMILES is absent from the training set, plus a separate uniqueness figure for redundancy within one batch of generations), and Lipinski compliance as a coarse synthesizability-adjacent proxy — for exactly this reason: optimizing one in isolation is not the goal.

7.2 Latent Space Models

Variational Autoencoders (VAEs) approach generation by learning a continuous latent space that a decoder can sample from. An encoder $q_\phi(z \mid x)$ maps a molecule x to a distribution over a latent vector z; a decoder $p_\theta(x \mid z)$ reconstructs a molecule from z; and training jointly minimizes reconstruction error and the KL divergence between $q_\phi(z \mid x)$ and a fixed prior, usually $\mathcal{N}(0, I)$:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}\left(q_\phi(z \mid x) \,\|\, p(z)\right)$$

The KL term is what makes the latent space usable for generation, not just compression: it pulls every training molecule's encoded distribution toward the same shared prior, so that after

training, a z sampled directly from $\mathcal{N}(0, I)$ — with no corresponding input molecule at all — decodes to something the decoder has plausibly seen a similar-looking (nearby-in-z) example of during training, rather than landing in an unexplored, poorly-decoded region of latent space. Gómez-Bombarelli et al. (2018) built the first molecular VAE along exactly these lines, encoding and decoding SMILES strings directly, and demonstrated a second capability forward-only models cannot offer at all: because z is continuous, gradient-based optimization *within the latent space* — moving z in the direction a separately-trained property predictor's gradient suggests — can search for molecules with a target property, entirely by decoding points the model was never explicitly shown.

Their original decoder, working directly in SMILES, paid the exact cost Chapter 2 (Section 2.1) described: a decoder has to implicitly learn SMILES' syntax rules (balanced ring-closure digits, valid branch nesting) well enough to avoid generating strings that fail to parse at all, and a sampled latent point decoded through an imperfectly-trained model has no guarantee of doing so. Pairing a molecular VAE with **SELFIES** (Krenn et al., 2020) instead — Chapter 2's forward reference to this chapter — removes that failure mode structurally rather than statistically: because every string SELFIES' grammar can produce decodes to a valid molecule, a decoder emitting SELFIES tokens cannot produce a syntactically invalid output no matter how undertrained it is, which is exactly the "100% validity" property Chapter 2 named.

7.3 Sequence-based Auto-regressive Models

An alternative to encoding a whole molecule into one latent vector at once is to generate it one token at a time, autoregressively: $p(x) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$. The **Transformer** (Vaswani et al., 2017) is the now-standard architecture for this — a stack of self-attention layers that, for generation, must be made **causal**: token t's representation is computed attending only to tokens $1, \ldots, t$, never to tokens after it, so that at generation time — where tokens $t+1, \ldots, T$ genuinely do not exist yet — the model is never relying on information it will not actually have. Section 7.6's generator is built directly on this design: a stack of causally-masked self-attention layers over embedded SELFIES tokens, trained by next-token cross-entropy exactly as $p(x_t \mid x_{<t})$ above specifies.

Two named molecular applications of Transformers illustrate two different jobs the same underlying architecture can do, and it matters which one a given model performs. **ChemBERTa** (Chithrananda et al., 2020) is *encoder*-style (following BERT): trained with masked-token prediction on 77 million PubChem SMILES, it produces representations useful for downstream property prediction (a Chapter 5/6-style forward task), not for generating new molecules — it has no autoregressive generation mechanism at all. (This is also the one citation in this chapter that is not published in a peer-reviewed venue as far as verifiable directly from its own listing — an arXiv preprint, cited here because the outline names it specifically, with that status disclosed rather than implied otherwise.) **MolGPT** (Bagal et al., 2022), by contrast, is *decoder*-style (following GPT): trained with exactly the causal next-token objective above, it generates molecules token by token, optionally conditioned on target properties. Section 7.6's generator follows MolGPT's decoder-only design, not ChemBERTa's encoder-only one — the distinction is not cosmetic, since only the former can generate anything at all.

7.4 3D Diffusion Models

Sections 7.2-7.3 both generate a molecular graph or string — Chapter 2′s 2D constitution, in Chapter 6′s terms — with no reference to 3D shape. **Diffusion models** extend generation into 3D directly: a forward process progressively adds Gaussian noise to a molecule's atomic coordinates (and, in the joint formulations, its atom types) over T steps until they are indistinguishable from noise, and a neural network is trained to reverse that process step by step, learning to denoise. Sampling then starts from pure noise and runs the learned reverse process to produce a full 3D structure.

Because atomic coordinates are what is being generated, the denoising network faces exactly the consistency requirement Chapter 6 (Section 6.4) formalized: a molecule's identity does not depend on the arbitrary orientation its coordinates happen to be stored in, so the denoiser must be **equivariant** — rotate the noisy input, and the predicted noise (equivalently, the denoised structure) must rotate identically. Hoogeboom et al. (2022) built exactly this — Equivariant Diffusion for Molecules (EDM) — using an E(3)-equivariant network architecture in the role Chapter 6 described SchNet, EGNN, and NequIP filling for property prediction, here repurposed as the reverse-diffusion denoiser instead. This is a direct, concrete example of Chapter 6′s closing point that its equivariant architectures "become directly load-bearing once 3D structure is actually on the table" — diffusion is one of the tables it gets used on.

Section 7.6′s hands-on project does not implement a 3D diffusion model, for the same reason Chapter 6′s equivariant networks stayed theory-only: EGFR's real bioactivity data, as retrieved from ChEMBL, is 2D structures (SMILES) with no bundled 3D conformers, and training a diffusion model that actually needs 3D training targets is out of this chapter's scope. Chapter 11′s DiffDock (Corso et al., 2023, covered there in full) is the closest relative actually implemented in this book — a diffusion model over docking *poses* specifically, generating a ligand's position and orientation relative to a fixed protein rather than generating novel ligand connectivity from scratch, a narrower and distinct problem from full 3D de novo generation.

7.5 Reinforcement Learning (RL)

Pretraining (Sections 7.2-7.3) teaches a generative model to imitate a training distribution — sample from a pretrained EGFR-focused generator, and it produces things that look like the EGFR-active compounds it was shown. It has no mechanism for producing things that score *better* than that training distribution on a target property, because nothing in a maximum-likelihood pretraining objective ever asks it to. **Reinforcement learning** closes that gap by treating sequence generation itself as a decision process: the policy is the generative model π_θ, an "action" is a complete generated sequence, and a scalar **reward** $R(x)$ — computed after the fact, from whatever downstream property actually matters — replaces the training-set likelihood as the training signal.

REINFORCE (Williams, 1992) is the foundational policy-gradient estimator for this setting:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \pi_\theta}[R(x)\, \nabla_\theta \log \pi_\theta(x)]$$

— increase the log-probability of sequences in proportion to how much reward they earned, decrease it for sequences that earned little or none, estimated from Monte Carlo samples drawn from the policy itself rather than requiring the reward to be differentiable at all. That last property is exactly why RL, rather than ordinary gradient descent, is the right tool here: Section 7.6′s reward — a trained classifier's predicted probability, composed with a discrete Lipinski pass/fail — is not a differentiable function of the generator's parameters, so backpropagating "reward" directly through the sampling step is not an option; REINFORCE's log-derivative trick is what makes optimizing it possible at all. A well-known practical refinement, used in this chapter's implementation, subtracts a **baseline** (here, a running mean of recent rewards) from $R(x)$ before scaling the gradient: since $\mathbb{E}_{x\sim\pi_\theta}[b\,\nabla_\theta \log \pi_\theta(x)] = 0$ for any x-independent b, this leaves the gradient estimator unbiased while substantially reducing its variance, because the learning signal now reflects whether a sample did *better or worse than recently typical*, not its raw, noisier reward magnitude.

Proximal Policy Optimization (PPO; Schulman et al., 2017) is the modern, more stable alternative used throughout contemporary RL practice (including large language model fine-tuning): it clips the policy update to prevent any single step from moving π_θ too far from the policy that generated the current batch of samples, permitting multiple gradient steps per batch of sampled data where REINFORCE affords only one. Section 7.6 implements REINFORCE specifically, not PPO — a deliberate scope decision, named here rather than substituted silently: REINFORCE's update is a direct, few-line translation of the equation above, keeping the connection between theory and code legible, at the cost of PPO's improved sample efficiency and training stability. Olivecrona et al. (2017) is the direct precedent for applying this exact idea — REINFORCE fine-tuning of a pretrained sequence generator — to molecule design specifically, optimizing an RNN-based SMILES generator against predicted bioactivity; Section 7.6 follows the same fine-tune-a-pretrained-generator-with-a- learned-reward structure, substituting a Transformer generator over SELFIES tokens for their RNN over SMILES characters.

Multi-objective reward composition — this section's other named topic — is handled in Section 7.6 by the simplest approach available: a weighted sum, $R(x) = w_1 \cdot P(\text{active} \mid x) + w_2 \cdot \mathbb{1}[\text{Lipinski pass}]$, combining a bioactivity term (the Chapter 5-style reward oracle, Section 7.6) with an ADMET-proxy term (Chapter 1′s Lipinski's Rule of Five, reused directly rather than reimplemented). Olivecrona et al. (2017) use a comparably simple composition for the same reason it is used here: a weighted sum is transparent about the trade-off it encodes (the weights *are* the trade-off, stated explicitly rather than left implicit) even though it cannot represent every possible preference over the two objectives — more sophisticated multi-objective RL formulations exist but add complexity this chapter's scope does not require.

7.6 Hands-on Project: Generative Transformer + RL for De Novo EGFR Inhibitor Design

The project code lives in this chapter's folder (`ch07_generative_transformer_rl/`) and implements the full pipeline this chapter has been building toward: pretrain a Transformer generator on real EGFR-active compounds, then fine-tune it with REINFORCE against a reward built from a Chapter 5-style bioactivity oracle and Chapter 1′s Lipinski's Rule of Five.

Data and the reward oracle

`gen_transformer.py` extracts real IC50 bioactivity records for EGFR (`CHEMBL203` — the running target since Chapter 1′s `fetch_data.py` and Chapter 3′s `structural_features.py`) from the ChEMBL API, using the same extraction, standardization (Chapter 4, Section 4.1), and median-deduplication pipeline as Chapters 4-5. Each compound is labeled active/inactive at a 1 μM IC50 cutoff — a deliberate, explicit modeling choice for this project specifically, in the same spirit as (and using the same threshold-selection reasoning as) Chapter 5′s hERG threshold, not a value taken from a specific external paper's methodology.

A `RewardOracle` — an XGBoost classifier on Chapter 2′s ECFP4 fingerprints, structurally identical to Chapter 5′s `herg_qsar.py` model — is trained on this data and evaluated with a Bemis-Murcko scaffold split (Chapters 5-6) rather than a random split, so its reported accuracy is an honest estimate of how well it generalizes to chemical series it was not trained on — exactly the property that matters here, since its job is to score molecules the generator invents that were, by construction, never in its own training set either.

The generator

Active compounds' SMILES are converted to SELFIES (Krenn et al., 2020) and tokenized into a vocabulary built directly from the symbols actually observed in the corpus (Section 7.2-7.3′s representation choice, made concrete). `GenerativeTransformer` is a small decoder-only Transformer (Section 7.3): token and learned positional embeddings, 4 causally-masked self-attention layers (`d_model=128`, 4 heads), and a linear output head producing next-token logits over the vocabulary — `torch.nn.TransformerEncoder` with a causal attention mask, which is architecturally a Transformer *decoder* in the generation sense Section 7.3 defines (no cross-attention to a separate encoder sequence exists to attend to, since this is unconditional, not sequence-to-sequence, generation).

```
class GenerativeTransformer(nn.Module):
    def forward(self, ids):
        x = self.token_embedding(ids) + self.position_embedding(pos)
        causal_mask = torch.triu(torch.full((T, T), float("-inf")), diagonal=1)
        x = self.blocks(x, mask=causal_mask, is_causal=True)
        return self.head(self.ln_out(x))
```

The model is pretrained by ordinary next-token cross-entropy (Section 7.3) on the active-compound corpus, then sampled from and RL fine-tuned via `reinforce_finetune` (Section 7.5): each iteration samples a batch of sequences with gradient-tracked log-probabilities (`sample_with_logprobs`), decodes and scores them (`score_sequences` — SELFIES to SMILES, RDKit parse, oracle probability, Lipinski check), and takes one REINFORCE policy-gradient step against a running-mean baseline.

Running it and reading the output

```
cd ch07_generative_transformer_rl
pip install -r requirements.txt
python gen_transformer.py --use-cached-raw
```

Running the default configuration (offline, deterministic, seed 0) against the bundled fixture cleans 3,000 raw IC50 records to 1,508 unique standardized compounds (860 active / 648 inactive at the 1 μM threshold). The reward oracle reaches **84.1% accuracy / 0.896 ROC-AUC** on a 302-compound scaffold-split held-out set (1,206 train) — a strong enough signal to be worth optimizing against, and, per Chapter 5′s own lesson, one whose accuracy is being reported honestly rather than inflated by a random split. The generator is pretrained for 20 epochs on the resulting 842-compound active corpus (vocabulary size 42), reaching a final cross-entropy loss of 0.787.

Sampling 200 molecules from the pretrained-only policy, before any RL:

Metric	Before RL	After 25 REINFORCE iterations
Valid fraction	1.000	1.000
Unique fraction (of valid)	1.000	0.960
Novel fraction (of valid)	0.990	0.970
Mean predicted P(EGFR-active)	0.120	0.405
Lipinski-compliant fraction	0.995	1.000
Mean reward	0.383	0.583

Three things are worth reading precisely here, not just the headline number. First, **validity never moves, because it cannot** — every sampled sequence decodes through SELFIES' guaranteed-valid grammar (Section 7.2), so 100% validity is a property of the representation, not a result RL achieved; the number is reported to make that point concretely rather than to claim credit for it. Second, **mean predicted activity more than triples** (0.120 ⟶ 0.405) after only 25 REINFORCE iterations, with novelty and uniqueness both still above 96% — the generator is not collapsing onto a handful of memorized or repeated high-reward outputs, at least not within this run's iteration budget, though 25 iterations is a modest budget and this is not a guarantee that stays true with substantially longer fine-tuning (see Limitations, below). Third, **Lipinski compliance also improves** (0.995 ⟶ 1.000) even though it was already near-ceiling before RL — a small, genuine effect of including it in the reward at all, not a large discovery.

Reproducibility

Dependencies are version-floored (`torch>=2.2`, `rdkit>=2023.9.1`, `xgboost>=2.0`, `selfies>=2.1` in `requirements.txt`, validated against torch 2.13.0, rdkit 2026.3.5, xgboost 3.4.1, and selfies 2.1.1). `data/raw_egfr_bioactivities_sample.json` bundles a real 3,000-record extract (fetched 2026-08-20) so the full pipeline runs offline and deterministically with `--use-cached-raw`, following Chapters 4-5′s resilience pattern. The results above took 13 minutes on a single CPU core, dominated by structure standardization and the RL loop's autoregressive sampling — this implementation recomputes the full causal attention forward pass at every generated token rather than caching key/value projections from previous steps, an explicit, disclosed simplification that keeps the sampling code's connection to Section 7.3′s equations direct, at a real, measured cost in wall-clock time. The 25-test suite in `tests/test_gen_transformer.py` checks data cleaning, SELFIES tokenization round-trips, the reward oracle, both the plain and gradient-tracked sampling paths,

and a short, synthetic-molecule end-to-end pretrain-then-RL run. Most individual tests are fast; the suite as a whole takes about 3 minutes, dominated by the handful of tests and the one CLI subprocess check that standardize a real slice of the bundled fixture and fit a fresh reward oracle rather than reusing one — module-scoped pytest fixtures share that work across tests where possible, but the CLI check necessarily runs as its own process and repeats it. One test calls the live ChEMBL API directly. `pip install -r requirements-dev.txt && pytest` reproduces all 25 results.

Limitations and what comes next

This implementation cuts several corners relative to a production RL molecule-design pipeline, each named here rather than left implicit. The reward is only as good as the oracle producing it — an 84.1% accurate classifier, not ground truth — so a high reward is evidence a molecule is *predicted* active, not evidence it actually would be; a sufficiently RL-optimized generator can, in principle, learn to exploit oracle blind spots rather than genuinely useful chemistry, a failure mode the literature calls reward hacking, which this chapter's short, 25-iteration fine-tuning run does not run long enough to clearly exhibit but does not structurally prevent either. Olivecrona et al. (2017) and most production RL-for-generation pipelines add a KL-divergence penalty against the pretrained policy specifically to slow this kind of drift; this implementation omits it, trading some robustness against longer fine-tuning runs for a simpler, more directly auditable REINFORCE loss (Section 7.5). And every molecule this pipeline proposes is still, in the end, a 2D structure scored by a 2D-fingerprint classifier — it says nothing about whether the molecule would actually bind EGFR's ATP pocket in the 3D sense Chapter 3 characterized structurally, which is precisely the question Chapter 11′s docking methods, and the true protein-ligand binding affinities they estimate, are built to answer. Generated candidates from a pipeline like this one are a hypothesis worth docking and, eventually, synthesizing and testing — not a finished answer.

A note on Google Colab

Colab's default runtime preinstalls `torch` but not `rdkit`, `xgboost`, or `selfies`; run `!pip install rdkit xgboost selfies` in the first cell. No GPU is required, though training will run faster on one if selected.

References

- Sanchez-Lengeling, B., & Aspuru-Guzik, A. (2018). Inverse molecular design using machine learning: Generative models for matter engineering. *Science*, 361(6400), 360–365. https://doi.org/10.1126/science.aat2663
- Gómez-Bombarelli, R., Wei, J. N., Duvenaud, D., Hernández-Lobato, J. M., Sánchez-Lengeling, B., Sheberla, D., Aguilera-Iparraguirre, J., Hirzel, T. D., Adams, R. P., & Aspuru-Guzik, A. (2018). Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2), 268–276. https://doi.org/10.1021/acscentsci.7b00572
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30 (NeurIPS 2017). arXiv:1706.03762 (no DOI; NeurIPS proceedings papers from this era were not assigned one).

- Chithrananda, S., Grand, G., & Ramsundar, B. (2020). ChemBERTa: Large-scale self-supervised pretraining for molecular property prediction. arXiv:2010.09885. Unlike this chapter's other citations, this is an arXiv preprint with no confirmed peer-reviewed publication as of this writing, verified directly against arXiv's own listing rather than assumed — cited because the outline names it specifically, with that status disclosed here rather than implied otherwise.
- Bagal, V., Aggarwal, R., Vinod, P. K., & Priyakumar, U. D. (2022). MolGPT: Molecular generation using a Transformer-decoder model. *Journal of Chemical Information and Modeling*, 62(9), 2064–2076. https://doi.org/10.1021/acs.jcim.1c00600
- Hoogeboom, E., Garcia Satorras, V., Vignac, C., & Welling, M. (2022). Equivariant diffusion for molecule generation in 3D. *Proceedings of the 39th International Conference on Machine Learning*, PMLR 162, 8867–8887 (no DOI; PMLR does not assign one).
- Corso, G., Stärk, H., Jing, B., Barzilay, R., & Jaakkola, T. (2023). DiffDock: Diffusion steps, twists, and turns for molecular docking. *11th International Conference on Learning Representations* (ICLR 2023). arXiv:2210.01776 (no DOI, as above) — introduced here only as a forward reference; covered in full in Chapter 11.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4), 229–256. https://doi.org/10.1007/BF00992696 (the original journal article — not the same-titled 1992 book-chapter reprint under a different DOI, checked directly against Crossref before use here to avoid citing the wrong record).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347 (no DOI, as above).
- Olivecrona, M., Blaschke, T., Engkvist, O., & Chen, H. (2017). Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1), 48. https://doi.org/10.1186/s13321-017-0235-x
- Brown, N., Fiscato, M., Segler, M. H. S., & Vaucher, A. C. (2019). GuacaMol: Benchmarking models for de novo molecular design. *Journal of Chemical Information and Modeling*, 59(3), 1096–1108. https://doi.org/10.1021/acs.jcim.8b00839

See Chapter 1′s references for Lipinski et al. (2001, Rule of Five) and Mendez et al. (2019, ChEMBL); Chapter 2′s for Krenn et al. (2020, SELFIES); and Chapter 5′s for Bemis & Murcko (1996, scaffold definition) — all reused here rather than re-listed. RDKit itself has no official journal publication; its maintainers' recommended citation is "RDKit: Open-source cheminformatics. https://www.rdkit.org" (confirmed directly from the project's own documentation), matching the convention established in Chapter 4′s references.

All dataset sizes, oracle metrics, and generation statistics cited in Section 7.6 were computed directly by running `gen_transformer.py` against the bundled EGFR fixture on 2026-08-20, not taken from a secondary source — see `data/raw_egfr_bioactivities_sample.json` and `gen_transformer.py` to reproduce.